

AWS AUTOMATION

VM Automation, Auto Scaling & Application Load Balancing

on Amazon Web Services (AWS)

Final Project Report

Field	Details
Project Title	AWS Automation — VM Deployment, Auto Scaling & Load Balancing
Prepared By	[Your Name]
Date	August 2026
Cloud Platform	Amazon Web Services (AWS)
Technologies Used	EC2, AMI, Launch Template, Auto Scaling Group, Application Load Balancer, Target Group, CloudWatch, IIS, RDP
Project Type	Cloud Infrastructure Automation

1. Abstract

This project demonstrates the design and implementation of a scalable, fault-tolerant web application infrastructure on Amazon Web Services (AWS). Starting from a single manually configured virtual machine, the project evolves into a fully automated environment where new servers are launched, scaled, and load-balanced without manual intervention. The core objective was to eliminate the single point of failure inherent in a one-server setup by introducing Auto Scaling and an Application Load Balancer, so that the application remains available and responsive even during traffic spikes or instance failures.

The project was carried out in four major phases: (1) provisioning and configuring a Windows EC2 virtual machine and deploying a web application on it, (2) capturing that configured server as a reusable Amazon Machine Image (AMI), (3) using that AMI inside a Launch Template to automatically spin up identical servers through an Auto Scaling Group, and (4) distributing incoming traffic across those servers using an Application Load Balancer and Target Group with continuous health checks. The result is a self-healing, horizontally scalable system that mirrors how production applications are deployed in the real world.

2. Project Objective

2.1 Why This Project Was Created

Most beginner cloud deployments start with a single EC2 instance running a web application directly. While this works for learning purposes, it does not reflect how real production systems are built, because a single server has no redundancy — if it crashes, gets overloaded, or needs maintenance, the entire application goes down. This project was created to move beyond that limitation and understand, hands-on, how AWS handles scalability and availability at an infrastructure level.

2.2 Goals Achieved

- Successfully launched and configured a Windows EC2 instance from scratch.
- Connected to the instance remotely using RDP and installed a web server (IIS) along with the web application.
- Converted the configured server into a reusable AMI so identical servers can be launched on demand.
- Built a Launch Template and connected it to an Auto Scaling Group with defined minimum, desired, and maximum instance counts.
- Configured dynamic scaling policies so the system automatically adds or removes servers based on load.
- Set up a Target Group and an Application Load Balancer to evenly distribute traffic across all healthy instances.
- Validated the entire pipeline by confirming health checks, instance registration, and successful traffic distribution.

3. Problem Statement

3.1 Limitations of a Single-Server Setup

A traditional deployment where one EC2 instance hosts the entire application suffers from several structural weaknesses:

- **Single Point of Failure:** if the instance crashes, stops responding, or the underlying hardware fails, the application becomes completely unavailable.
- **No Elasticity:** the server has a fixed capacity. During high traffic it becomes slow or unresponsive, and during low traffic the same resources are wasted.
- **Manual Recovery:** bringing the application back after a failure requires a person to notice the issue and manually fix or relaunch the server, causing downtime.
- **Downtime During Updates:** patching, scaling, or maintaining the server means taking the whole application offline.

3.2 The Need for Scalability and High Availability

To overcome these problems, real-world systems are designed to run on multiple servers simultaneously, spread across a pool that can grow or shrink automatically. This introduces two critical properties: scalability, the ability to add or remove compute capacity in response to demand, and high availability, the ability to keep the application running even if individual components fail. This project implements both properties using AWS-native services — Auto Scaling for scalability and an Application Load Balancer combined with health checks for high availability.

4. Architecture Overview

The diagram below summarizes the final architecture built in this project. A client request first reaches the Application Load Balancer, which forwards it to one of several healthy EC2 instances registered in a Target Group. These instances are members of an Auto Scaling Group that was built from a Launch Template referencing a custom AMI. Amazon CloudWatch continuously monitors the instances and feeds metrics back to the Auto Scaling Group so it can add or remove servers as needed.

AWS Auto Sc

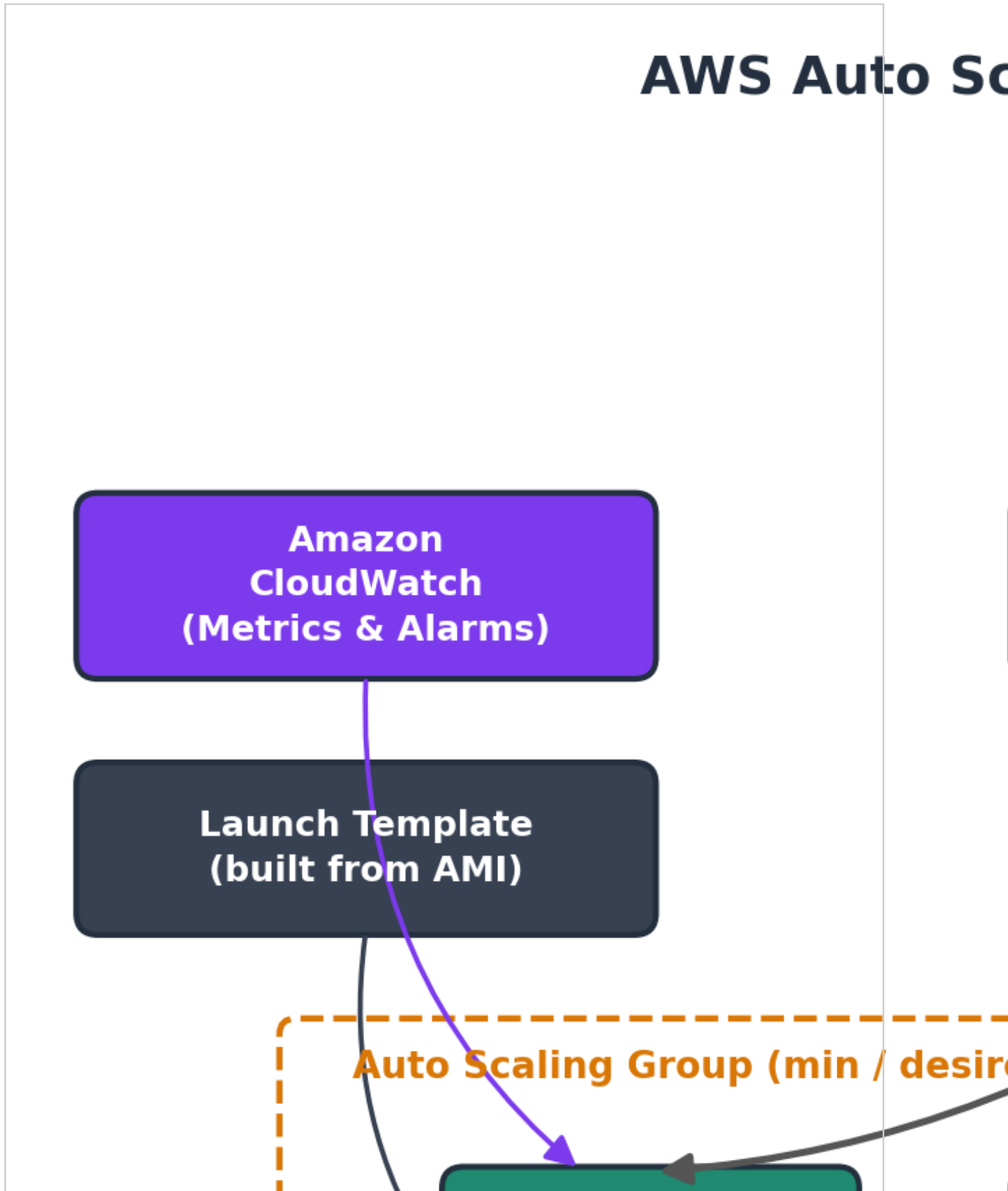


Fig. 4.1 — End-to-end architecture: Client → Load Balancer → Target Group → Auto Scaling Group of EC2 instances, monitored by CloudWatch.

4.1 Request Flow Explanation

- A user opens the application in a browser and sends an HTTP/HTTPS request over the internet.
- The request first arrives at the Application Load Balancer (ALB), which acts as the single public entry point.
- The ALB consults its associated Target Group to identify which EC2 instances are currently marked "healthy."
- The ALB forwards the request to one healthy instance using its routing algorithm, so no single server is overloaded.
- The EC2 instance (running IIS and the web application) processes the request and returns the response through the ALB back to the user.
- In the background, CloudWatch continuously tracks metrics such as CPU utilization; if load increases beyond the configured threshold, the Auto Scaling Group launches new instances from the Launch Template and automatically registers them with the Target Group — and removes them again once load decreases.

5. AWS Services Used

Service	Role in This Project
Amazon EC2	Provided the virtual machines (Windows Server) that host the web application.
Amazon Machine Image (AMI)	A snapshot of the fully configured EC2 instance, used to launch identical new servers.
Launch Template	A reusable configuration blueprint (AMI, instance type, security group, etc.) used by the Auto Scaling Group to create new instances.
Auto Scaling Group (ASG)	Automatically maintains the desired number of healthy EC2 instances and scales in/out based on demand.
Application Load Balancer (ALB)	Distributes incoming HTTP/HTTPS traffic across multiple EC2 instances for even load and high availability.
Target Group	A logical grouping of EC2 instances that the ALB routes traffic to, based on continuous health checks.
Amazon CloudWatch	Monitors instance metrics (CPU, network, etc.) and triggers scaling actions and alarms.

6. Theoretical Concepts

6.1 Auto Scaling — Complete Explanation

Auto Scaling is an AWS capability that automatically adjusts the number of running EC2 instances in a group to match the actual demand on an application, without any manual effort. Instead of provisioning a fixed number of servers based on guesswork, Auto Scaling lets the infrastructure grow when traffic increases and shrink when traffic decreases, so the application always has just enough capacity — not too little (which causes slowdowns) and not too much (which wastes money).

How Auto Scaling Works

An Auto Scaling Group (ASG) is built on top of a Launch Template (or Launch Configuration), which defines exactly what a new instance should look like — which AMI to boot from, the instance type, key pair, security groups, and storage. The ASG is then configured with three core numbers:

- Minimum size — the fewest instances that should ever be running, ensuring the application is never left with zero capacity.
- Desired capacity — the number of instances the ASG tries to maintain under normal conditions.
- Maximum size — the upper limit on how many instances can be launched, used to control cost and prevent runaway scaling.

Once these values are set, the ASG continuously monitors the health and load of its instances. If an instance fails a health check, the ASG automatically terminates it and launches a replacement from the Launch Template — this is what makes the system self-healing. If overall load rises above a defined threshold, the ASG launches additional instances (scaling out); if load falls, it terminates excess instances (scaling in).

Scaling Policies

- Manual Scaling — the desired capacity is changed by hand; useful for planned, predictable events.
- Scheduled Scaling — capacity changes automatically at specific times, useful when traffic patterns are predictable (e.g., business hours).
- Dynamic Scaling — capacity changes automatically and immediately in response to real-time metrics, such as CPU utilization crossing a threshold.
- Target Tracking Scaling — the most common dynamic policy, where you simply specify a target value (e.g., "keep average CPU at 50%") and AWS automatically calculates how many instances to add or remove to hold that target, similar to how a thermostat maintains a set temperature.

Benefits of Auto Scaling

- Elasticity: capacity automatically matches real demand instead of a fixed guess.
- Cost efficiency: you only pay for the instances that are actually needed at any given time.
- Fault tolerance: unhealthy instances are detected and replaced automatically, without manual restarts.
- Zero manual intervention: once configured, the system scales and heals itself continuously.

6.2 Load Balancing — Complete Explanation

A Load Balancer is a service that sits in front of a group of servers and distributes incoming network traffic evenly across them, instead of letting all requests hit a single server. This ensures no individual instance becomes a bottleneck, improves response times, and — critically — allows the underlying servers to be added or removed (by Auto Scaling) without ever changing the address the user connects to.

Types of AWS Load Balancers

- Application Load Balancer (ALB) — operates at Layer 7 (HTTP/HTTPS); used in this project because it understands web traffic and can route based on URL path, host header, etc.
- Network Load Balancer (NLB) — operates at Layer 4 (TCP/UDP); used for extremely high-performance, low-latency traffic.
- Gateway Load Balancer (GWLB) — used to deploy and scale third-party virtual security appliances.

How the Application Load Balancer Works in This Project

- The ALB is assigned a stable public DNS name that acts as the single entry point for all users.
- It forwards requests to a Target Group, which is simply a named collection of EC2 instances that should receive traffic.
- The ALB continuously performs health checks — periodically sending a request to each registered instance and checking for a valid HTTP response. Instances that fail are marked "unhealthy" and immediately removed from rotation, so users never get routed to a broken server.
- Traffic is distributed across all currently healthy targets using a routing algorithm (round robin by default), keeping load balanced as instances are added or removed by the Auto Scaling Group.

Benefits of Load Balancing

- High availability: if one server goes down, traffic is automatically shifted to the remaining healthy servers.
- Improved performance: no single server is overwhelmed, so response times stay consistent.
- Seamless scaling: works hand-in-hand with Auto Scaling — new instances are automatically registered and start receiving traffic without any DNS or client-side change.
- Single point of access: users always connect to one stable endpoint (the load balancer), while the actual number and identity of backend servers can change freely behind it.

6.3 Health Checks

A health check is a periodic test that the load balancer (and the Auto Scaling Group) performs against each instance to confirm it is functioning correctly — typically an HTTP request to a specific path, expecting a specific success response (e.g., HTTP 200). If an instance fails a configured number of consecutive checks, it is marked unhealthy: the ALB stops sending it traffic, and the ASG can terminate and replace it. This is the mechanism that turns a group of independent servers into a self-healing system.

6.4 High Availability

High Availability (HA) refers to designing a system so that it remains operational and accessible with minimal downtime, even when individual components fail. In this project, HA is achieved by running multiple EC2 instances across the Auto Scaling Group rather than relying on one server, combined with the Load Balancer's ability to instantly reroute traffic away from any instance that becomes unhealthy.

6.5 Fault Tolerance

Fault tolerance goes a step further than high availability — it means the system continues to function correctly even in the presence of a failure, ideally without the end user noticing anything happened. In this architecture, if one EC2 instance crashes, the Target Group's health check detects it within seconds, the ALB stops routing traffic to it, and the Auto Scaling Group launches a replacement instance from the same AMI — restoring full capacity automatically.

6.6 Dynamic Scaling

Dynamic scaling is the specific Auto Scaling behavior where the number of instances changes automatically and immediately in response to live, real-time metrics (rather than a fixed schedule). It relies on CloudWatch alarms that watch a metric such as average CPU utilization; when the metric crosses a defined threshold for a sustained period, a scaling action is triggered — either adding instances (scale-out) or removing them (scale-in).

6.7 Target Tracking Policies

A target tracking scaling policy is a simplified way to configure dynamic scaling: instead of manually defining thresholds and step sizes, you specify a single target value for a chosen metric (for example, "keep average CPU utilization at 50% across the group"). AWS then automatically calculates and applies the scaling adjustments needed to keep the metric close to that target — continuously adding or removing instances as required. This is the most commonly used scaling policy in production because it is simple to configure and self-adjusting.

7. Project Implementation

This section documents every step actually performed to build the project, along with the corresponding screenshots captured from the AWS Management Console during implementation.

Step 1: EC2 Instance Creation

The first step was to launch a Windows EC2 instance from the AWS Console. An appropriate Amazon Machine Image (Windows Server base image) and instance type were selected, a key pair was created for secure access, and a security group was configured to allow RDP (port 3389) and HTTP (port 80) traffic. Once configured, the instance was launched and its status was monitored until it reached the "Running" state.

The screenshot displays the AWS Management Console interface for the EC2 service. The left-hand navigation pane is open, showing the 'EC2' section with a list of resources including 'Instances', 'Instance Types', 'Launch Templates', 'Spot Requests', 'Savings Plans', 'Reserved Instances', 'Dedicated Hosts', 'Capacity Reservations', and 'Capacity Manager'. The 'Instances' resource is selected, and its sub-menu is expanded, showing 'Instances', 'Instance Types', 'Launch Templates', 'Spot Requests', 'Savings Plans', 'Reserved Instances', 'Dedicated Hosts', 'Capacity Reservations', and 'Capacity Manager'. The main content area shows the 'Instances (1/3)' page. At the top, there is a search bar and a 'Saved filter sets' dropdown. Below this is a table of instances. The table has columns for 'Name', 'Instance ID', and 'Instance state'. One instance is listed: 'webserver' with ID 'i-073afe79a2f9a00a3' and state 'Running'. Below the table, the details for the selected instance 'i-073afe79a2f9a00a3 (webserver)' are shown. The 'Details' tab is active, displaying the 'Instance summary' which includes the 'Instance ID' (i-073afe79a2f9a00a3), 'Public IPv4 address' (13.127.4.77), and 'Instance state' (Running). The bottom of the console shows the Windows taskbar with the search bar and various application icons.

Fig. 7.1 — Launching a new EC2 instance and configuring its basic settings in the AWS Console.

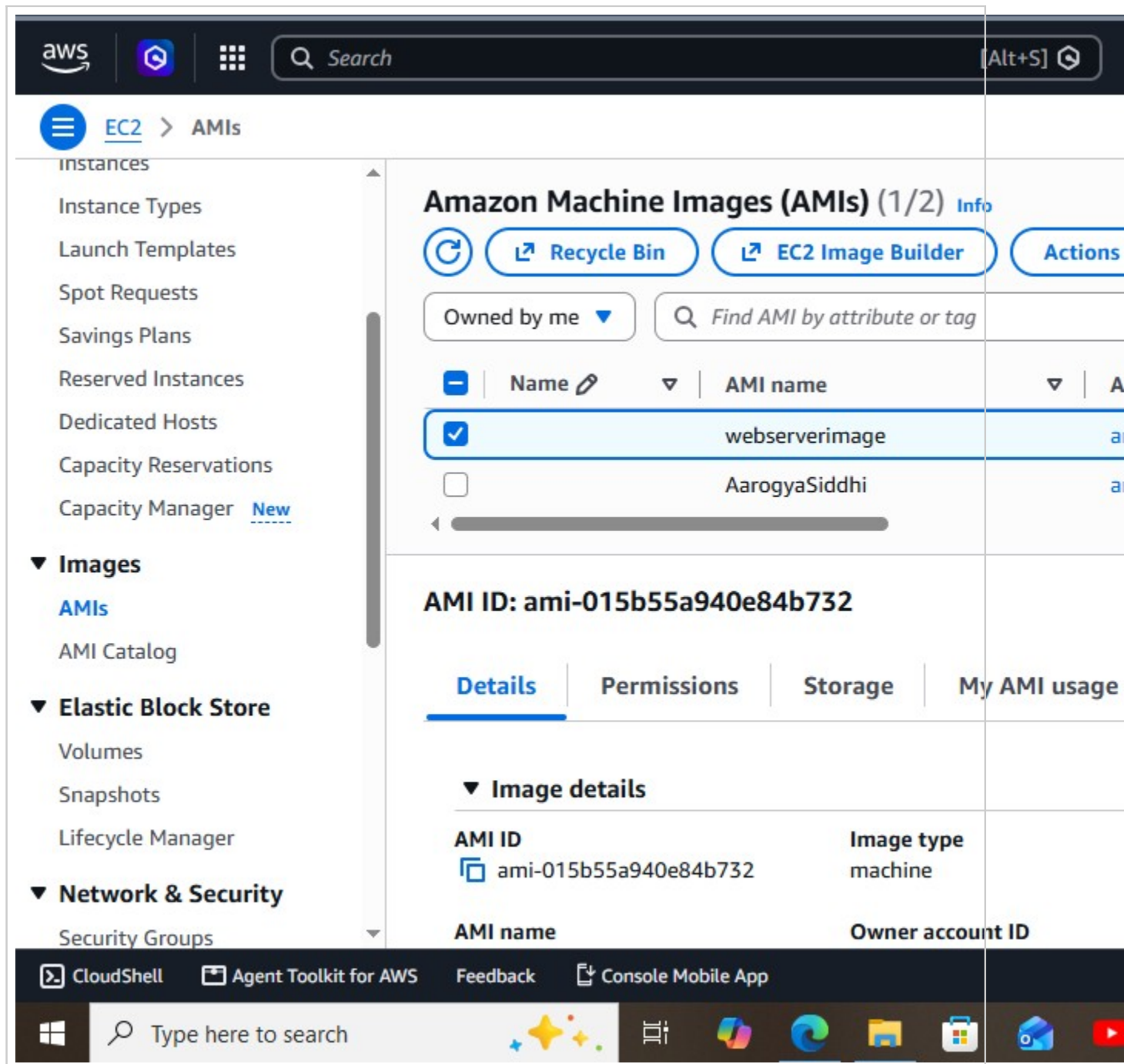


Fig. 7.2 — Configuring instance details such as instance type, key pair, and network/security group settings.

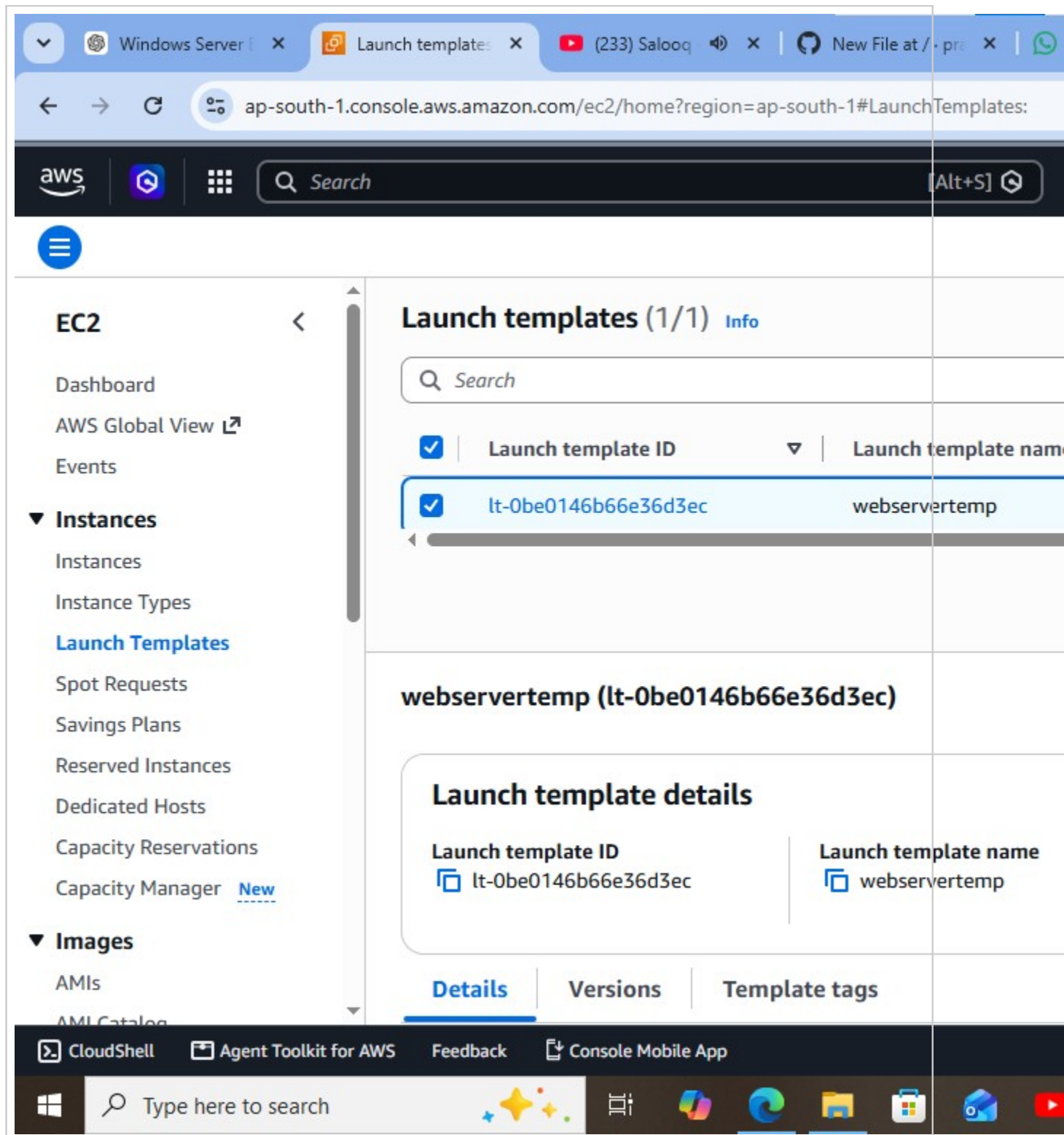


Fig. 7.3 — The EC2 instance successfully launched and showing a "Running" state in the Instances dashboard.

Step 2: RDP Connection to the VM

With the instance running, the Windows administrator password was retrieved using the associated key pair, and a Remote Desktop Protocol (RDP) session was established from the local machine to the EC2 instance's public IP address. This provided full desktop-level access to the server for further configuration.

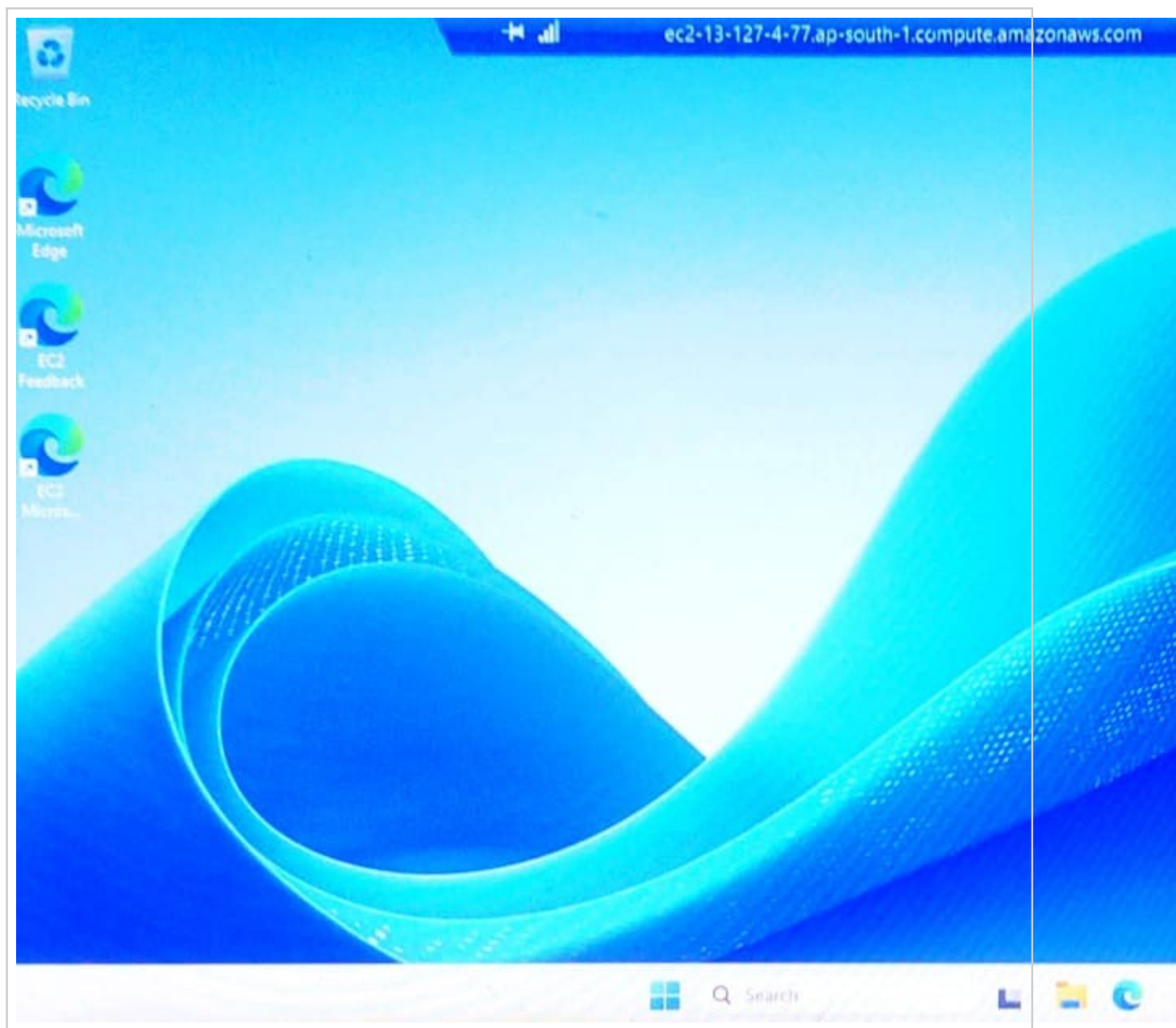


Fig. 7.4 — Connecting to the EC2 Windows instance using RDP with the retrieved administrator credentials.

Step 3: IIS Installation

After logging into the instance via RDP, Internet Information Services (IIS) was installed using Server Manager's "Add Roles and Features" wizard. IIS was chosen as the web server role responsible for hosting and serving the web application over HTTP.

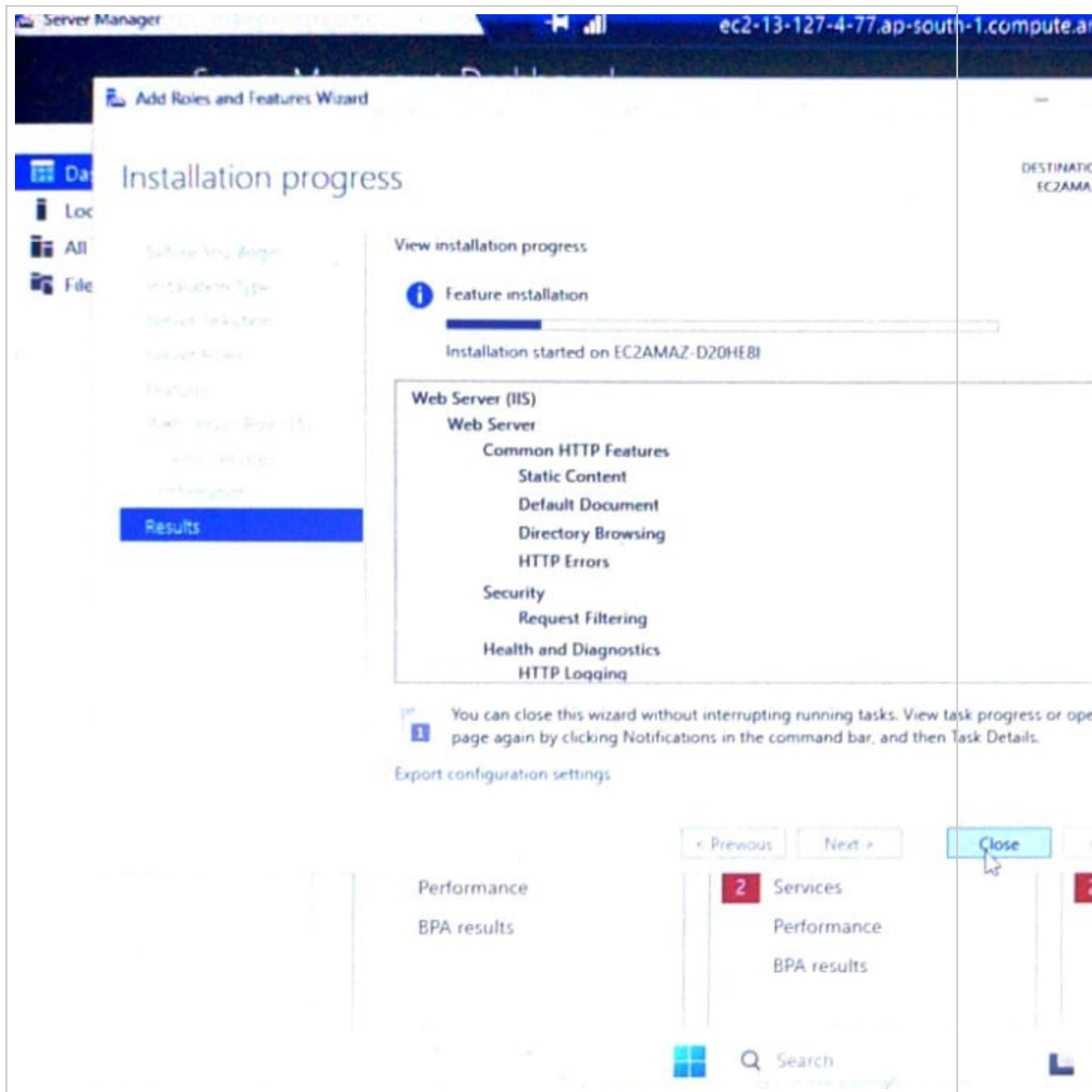


Fig. 7.5 — Installing the IIS (Web Server) role on the Windows EC2 instance via Server Manager.

Step 4: Web Application Deployment

Once IIS was installed and running, the web application files were placed inside the default IIS web root directory. The site was then verified by browsing to the instance's public IP address, confirming that the application was being served correctly from the VM.

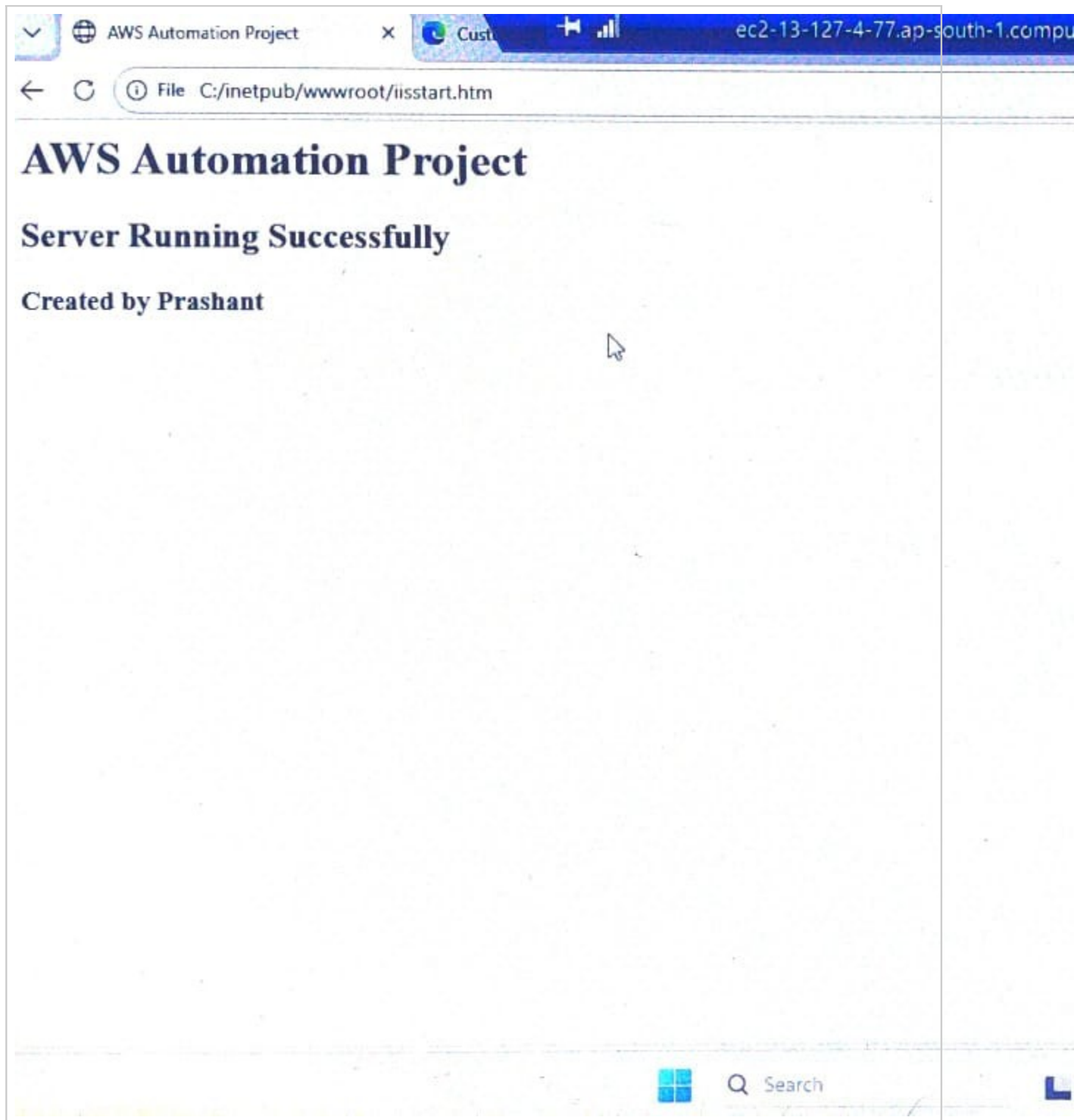


Fig. 7.6 — Verifying the deployed web application is being served correctly by IIS on the EC2 instance.

Step 5: AMI Creation

With the instance fully configured — operating system updated, IIS installed, and the application deployed — an Amazon Machine Image (AMI) was created from it. This captured the exact state of the server (OS, installed roles, and application files) as a reusable template, after which the original instance was terminated since it was no longer needed as a standalone server.

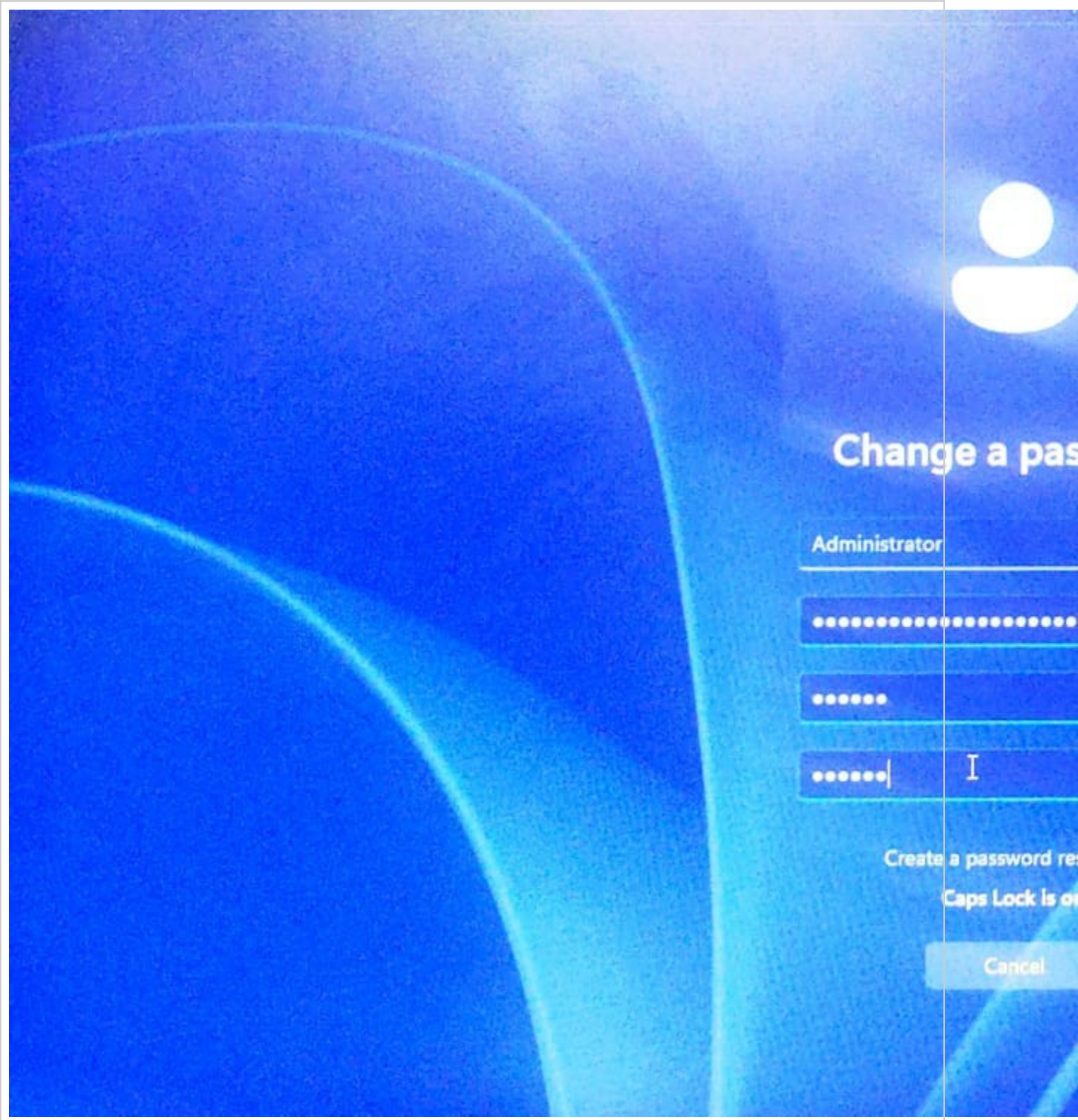


Fig. 7.7 — Creating a custom AMI from the fully configured EC2 instance.

Step 6: Launch Template Creation

A Launch Template was then created, referencing the custom AMI along with the instance type, key pair, and security group needed for new instances. This template acts as the single source of truth that the Auto Scaling Group uses whenever it needs to launch a new, identical server.

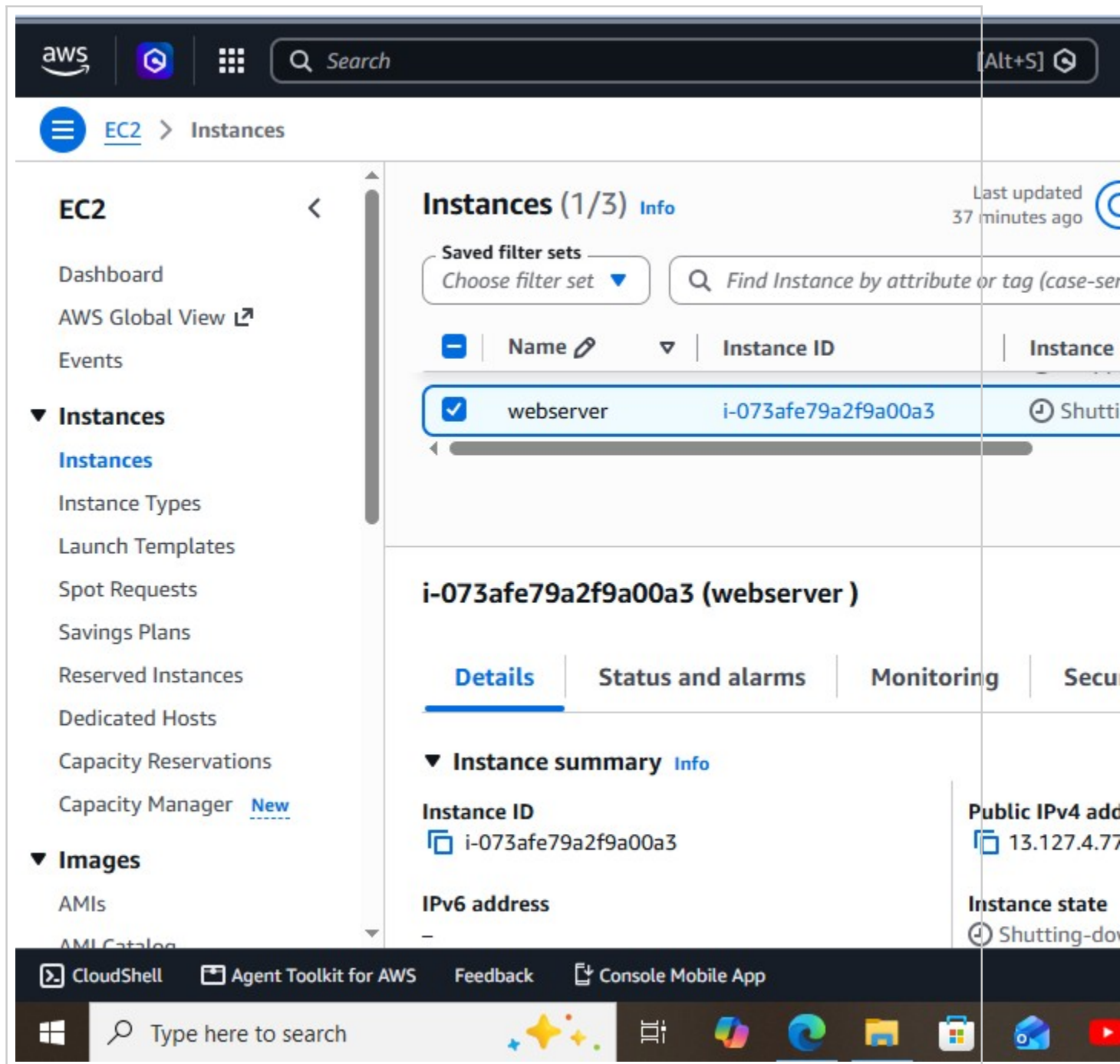


Fig. 7.8 — Creating a Launch Template based on the custom AMI, instance type, and security group.

Step 7: Auto Scaling Group Configuration

An Auto Scaling Group was created using the Launch Template, and the minimum, desired, and maximum instance counts were configured. A target tracking scaling policy was added so the group would automatically add or remove instances to keep average CPU utilization near a defined target.

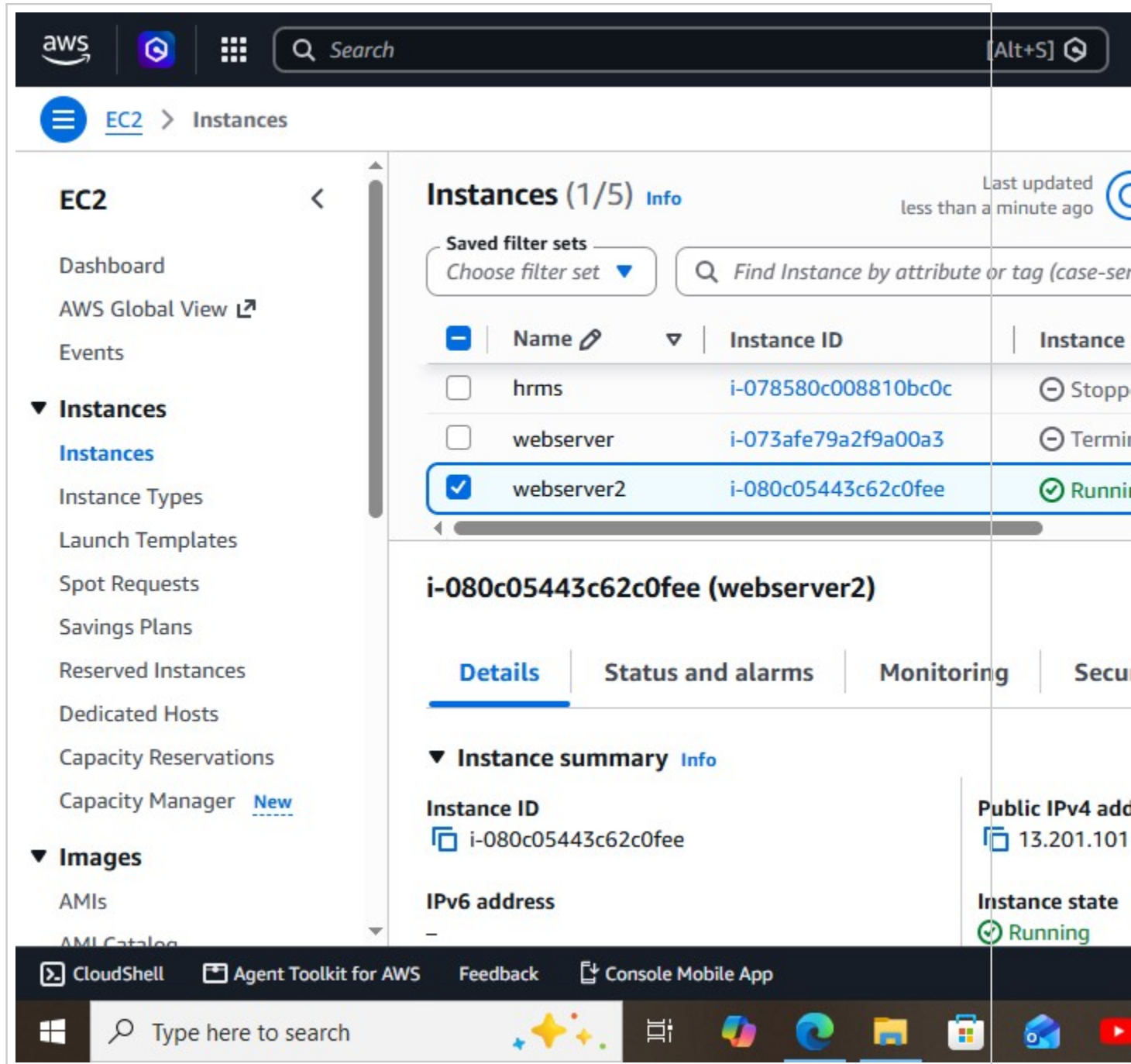


Fig. 7.9 — Configuring the Auto Scaling Group with minimum, desired, and maximum capacity, and a target tracking scaling policy.

Step 8: Target Group Configuration

A Target Group was created to define how the load balancer identifies and health-checks the backend instances. The health check path and thresholds (interval, healthy/unhealthy count) were configured so unresponsive instances would be detected quickly.

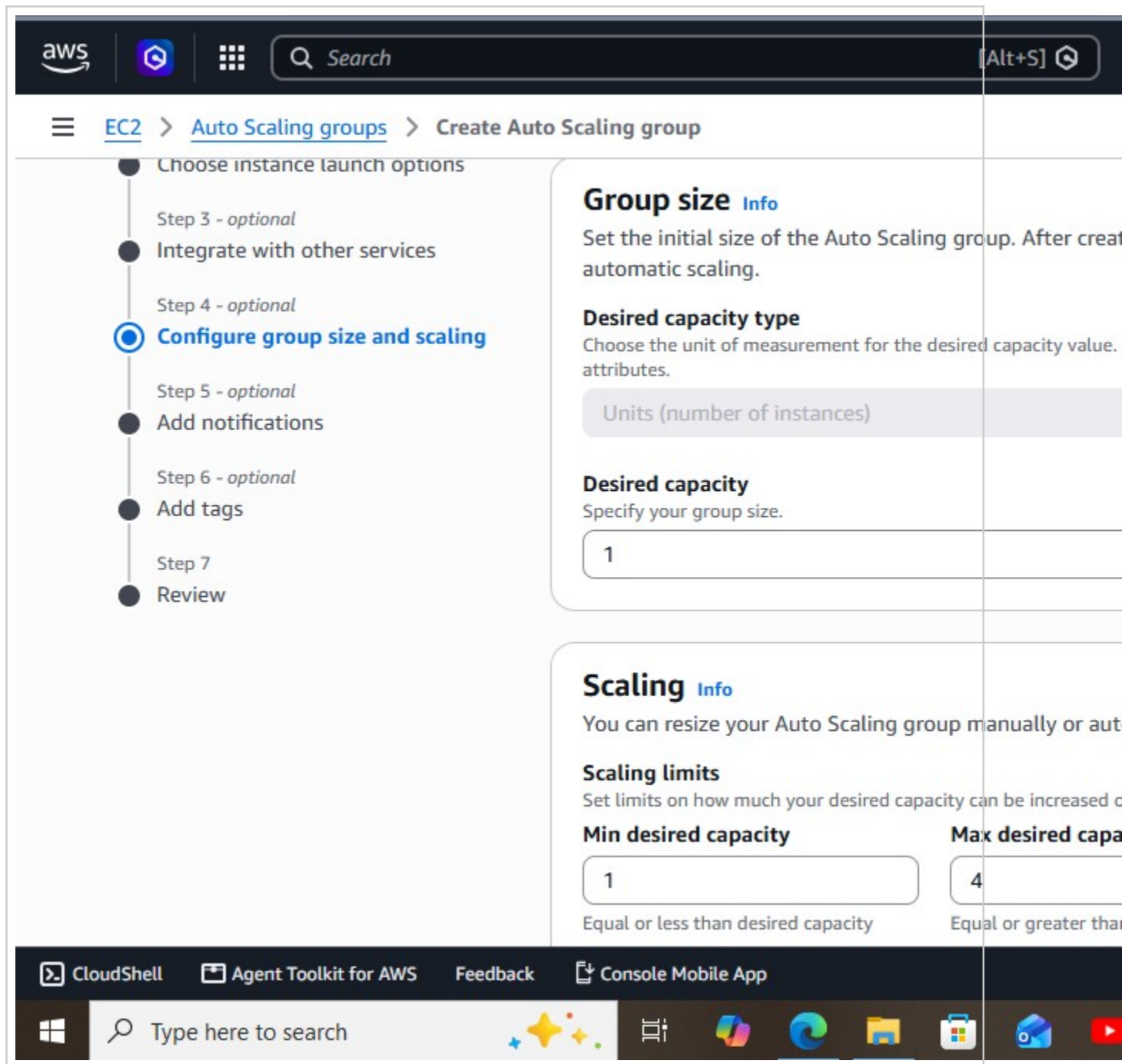


Fig. 7.10 — Creating a Target Group and configuring its health check settings.

Step 9: Application Load Balancer Configuration

An Application Load Balancer was provisioned and configured with a listener on port 80, forwarding traffic to the Target Group created in the previous step. The ALB was deployed across multiple Availability Zones for additional resilience.

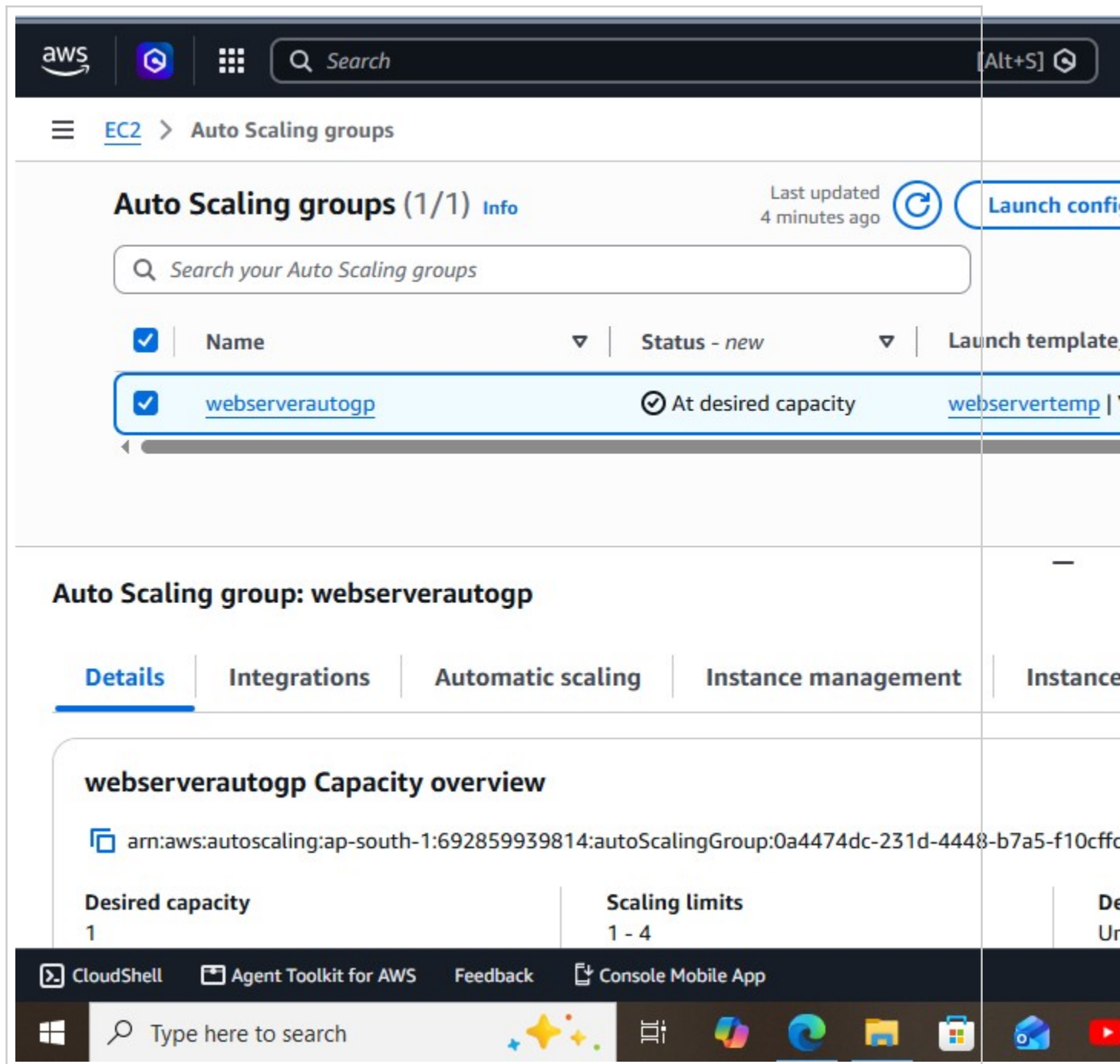


Fig. 7.11 — Configuring the Application Load Balancer and attaching it to the Target Group.

Step 10: Auto Scaling Group and Target Group Integration

The Auto Scaling Group was linked to the Target Group so that every instance launched by the ASG — whether at initial creation or during a later scale-out event — is automatically registered with the load balancer and immediately begins receiving traffic once it passes its health check.

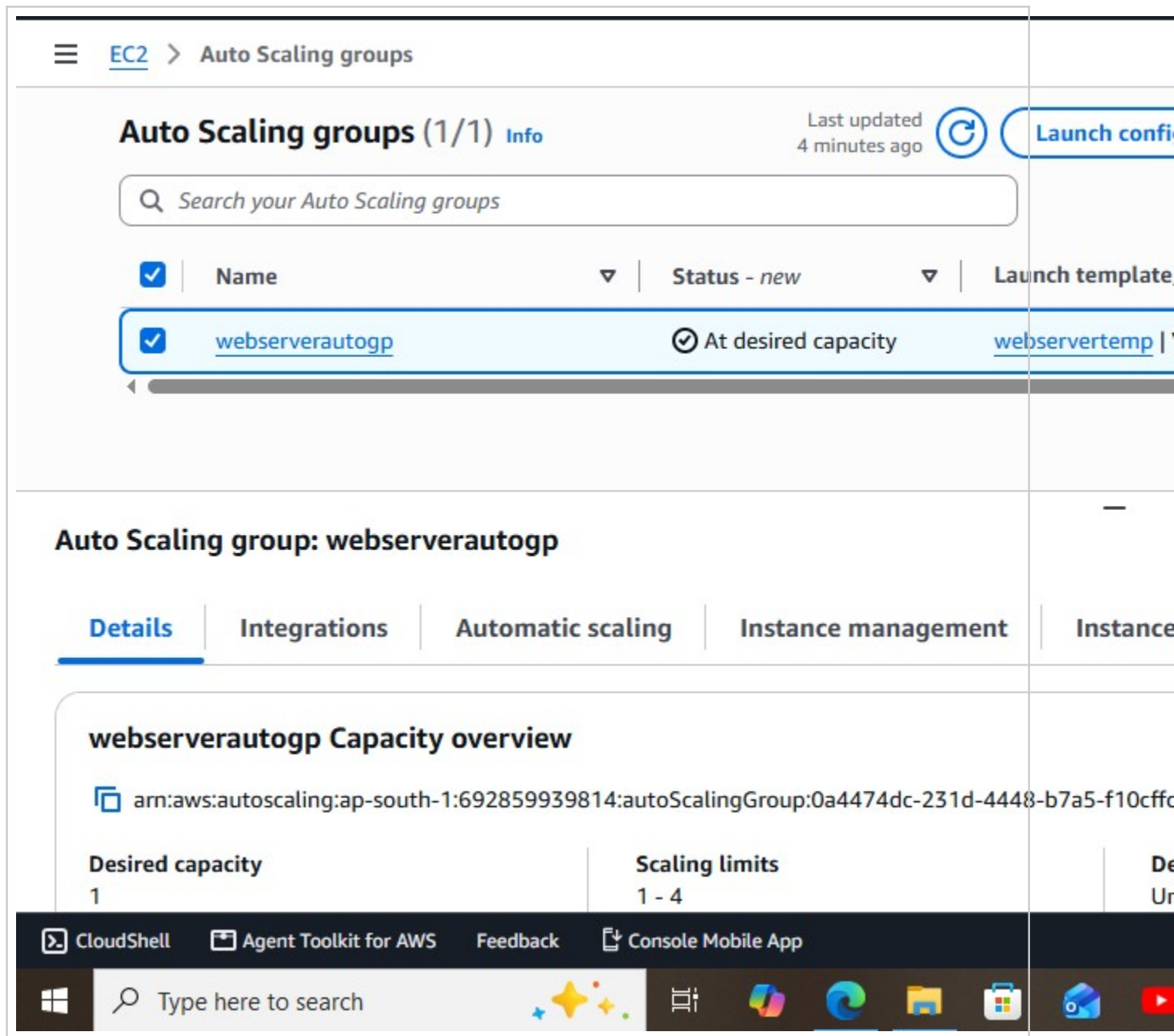


Fig. 7.12 — Attaching the Auto Scaling Group to the Target Group so new instances are auto-registered with the load balancer.

Step 11: Health Check Validation

Finally, the full pipeline was validated end-to-end: the Target Group console was checked to confirm that all registered EC2 instances showed a "healthy" status, and the ALB's public DNS name was accessed in a

browser multiple times to confirm that traffic was being served successfully and consistently across the running instances.

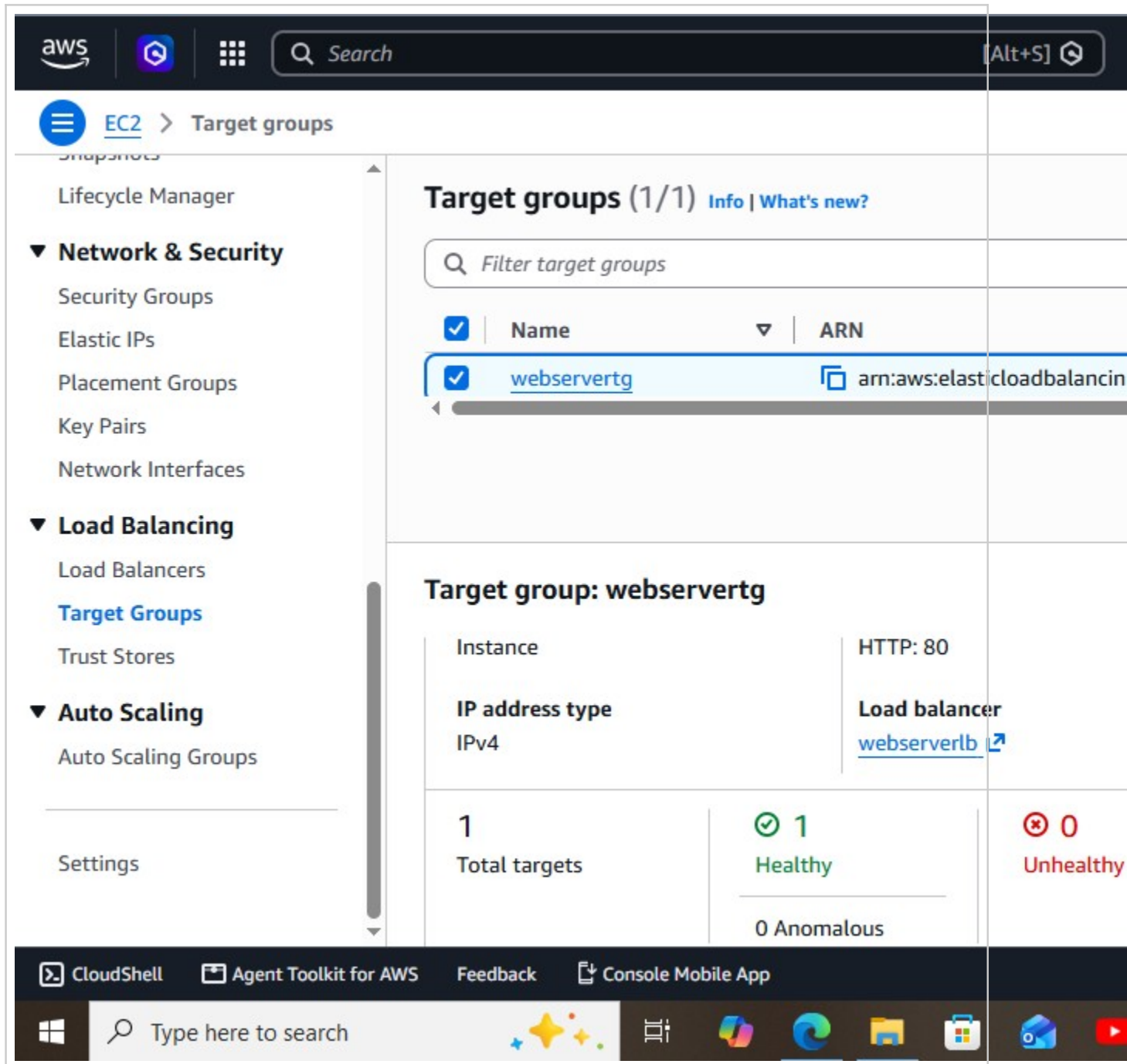


Fig. 7.13 — Validating that all EC2 instances behind the Target Group report a "healthy" status, confirming the pipeline works end-to-end.

8. Project Workflow (End-to-End Execution Flow)

- 1. Launch a Windows EC2 instance and configure security groups for RDP and HTTP access.
- 2. Connect to the instance via RDP using the retrieved administrator password.
- 3. Install IIS on the instance to serve as the web server.
- 4. Deploy the web application into the IIS web root and verify it locally via the instance's public IP.
- 5. Create a custom AMI from the fully configured instance and terminate the original instance.
- 6. Create a Launch Template referencing the custom AMI, instance type, key pair, and security group.
- 7. Create an Auto Scaling Group from the Launch Template with defined min/desired/max capacity and a target tracking policy.
- 8. Create a Target Group with an appropriate health check path and thresholds.
- 9. Create an Application Load Balancer with a listener that forwards traffic to the Target Group.
- 10. Attach the Auto Scaling Group to the Target Group so new instances are auto-registered.
- 11. Validate that all instances show "healthy" and that the ALB's DNS name serves the application correctly, confirming scalability and fault tolerance.

9. Challenges Encountered and Solutions

Challenge	Solution
AMI creation delay — the AMI took time to move from "pending" to "available" before it could be used in a Launch Template.	Waited for the AMI status to become "available" before proceeding, and used the console's refresh/status indicator to confirm readiness instead of assuming it was instant.
Instance registration — newly launched instances did not immediately appear as registered targets.	Confirmed the Auto Scaling Group was correctly attached to the Target Group and allowed the configured health check grace period to elapse before troubleshooting further.
Target Group attachment — initial traffic was not reaching instances due to a misconfigured target group/port mapping.	Re-verified that the Target Group's port and protocol matched the application (port 80/HTTP) and re-attached the Target Group to the Load Balancer listener.
Health check validation — some instances briefly showed "unhealthy" right after launch.	Adjusted the health check interval/threshold and confirmed the application was fully started before the first health check ran, resolving false negatives.
Security group access — RDP/HTTP access was initially blocked.	Updated the inbound rules of the security group to explicitly allow RDP (3389) and HTTP (80) from the required source IP ranges.

10. Real-World Industry Use Cases

The exact pattern implemented in this project — Auto Scaling Group + Load Balancer + Health Checks — is the same foundational pattern used by major technology companies to run services at massive scale:

Netflix: runs thousands of EC2 instances behind load balancers across multiple regions, using auto scaling to handle huge, unpredictable spikes in streaming demand (for example, when a popular show is released) without any manual server provisioning.

Amazon.com: the retail platform itself relies on auto scaling and load balancing to absorb massive traffic surges during sales events such as Prime Day and Black Friday, scaling capacity up in advance and back down afterward.

E-commerce platforms: generally use this same pattern so that checkout and product-browsing services stay responsive during flash sales, while keeping costs low during normal traffic periods.

Streaming platforms: use load balancers and auto scaling groups to distribute millions of concurrent viewer connections across many backend servers, while health checks ensure a failed server is replaced before it affects viewers.

11. Project Outcome

11.1 Final Results Achieved

- A fully working, self-healing infrastructure where EC2 instances are automatically launched, monitored, and replaced without manual effort.
- Traffic is evenly distributed across all healthy instances through the Application Load Balancer.
- The system can scale out automatically under increased load and scale in automatically once load decreases, optimizing both performance and cost.
- Unhealthy instances are detected within seconds and removed from rotation, with replacements launched automatically.

11.2 Scalability and Availability Benefits

- Elastic capacity: the application can absorb traffic spikes without manual intervention.
- No single point of failure: the loss of any one instance does not take the application offline.
- Cost optimization: resources scale down automatically during low-demand periods.
- Operational simplicity: once configured, the entire pipeline runs and heals itself continuously.

12. What I Learned

This project gave me hands-on, practical experience with the exact building blocks used to run scalable production systems on AWS. Beyond just clicking through the console, I came to understand why each component exists and how they connect to solve a real infrastructure problem.

- How to provision and secure an EC2 instance from scratch, including configuring security groups correctly for remote access and web traffic.
- How to convert a manually configured server into a reusable AMI, and why this is the foundation that makes automated, identical scaling possible.
- The relationship between a Launch Template, an Auto Scaling Group, and the min/desired/max capacity model, and how target tracking policies let AWS manage scaling automatically instead of relying on manual thresholds.
- How an Application Load Balancer and Target Group work together with health checks to keep traffic away from failing instances in real time.
- How to connect an Auto Scaling Group to a Target Group so that newly launched instances are automatically brought into service.
- The practical difference between high availability and fault tolerance, and how both are achieved together through redundancy, health checks, and automated replacement.
- Real troubleshooting skills — diagnosing issues like delayed AMI availability, target registration timing, and health check false negatives, which are common in real cloud deployments.

Overall, this project moved my understanding of AWS from "launching a single server" to actually designing a small but complete, production-style architecture — one that can grow, shrink, and heal itself automatically, which is exactly how modern cloud applications are expected to behave.

13. Conclusion

This project successfully demonstrates how a fragile, single-server deployment can be transformed into a resilient, self-scaling, and highly available system using core AWS services — EC2, AMI, Launch Templates, Auto Scaling Groups, Target Groups, and an Application Load Balancer. By automating VM provisioning through AMIs and Launch Templates, and by combining Auto Scaling with Load Balancing and continuous health checks, the resulting architecture can absorb traffic spikes, automatically recover from instance failures, and scale cost-efficiently — without any manual server management. This mirrors the exact approach used by real-world platforms serving millions of users, and provides a strong practical foundation for working with cloud infrastructure automation.